TD Clustering – Compte rendu

NGUYEN Dinh Huy NGUYEN Vinh Tri

Data Mining – INSA Centre Val de Loire – Novembre 2026

Table des matières

Organisation du rendu et utilisation du code **1**

1 Clustering k-means **2**
1.1 Exercice 1 : implémentation de k-means 2
1.2 Exercice 2 : tests sur le jeu de données [[2], [4], [6], [12], [24], [30]] 2
1.3 Exercice 3 : visualisations 2D 3
1.4 Exercice 4 : comparaison avec SciKit-Learn 4

2 Clustering hiérarchique **5**
2.1 Matrice de distance 5
2.2 Matrice de similarité 6

3 Étude de cas **6**
3.1 Exercice 1 : préparation des données 6
3.2 Exercice 2 : clustering et interprétation 7

Organisation du rendu et utilisation du code

L'archive contient :
— `section1_kmeans.py` : notre k-means (Section 1, Exercices 1 à 3) ;
— `section1_sklearn.py` : la comparaison avec SciKit-Learn (Section 1, Exercice 4) ;
— `section1_verifications.py` : la comparaison de notre affichage avec celui de l'énoncé (Section 1, Exercice 1) ;
— `section2_hierarchique.py` : le clustering hiérarchique (Section 2) ;
— `section3_etude_de_cas.py` : l'étude de cas (Section 3) ;
— `data/Country-data.csv` : le jeu de données de l'étude de cas ;
— `figures/` : les figures générées par les scripts ;
— `requirements.txt`, `README.md`, et `rapport/` qui contient ce compte rendu et son source LaTeX.

Installation. Notre distribution Linux n'autorise pas `pip install` en dehors d'un environnement virtuel, nous avons donc installé les bibliothèques ainsi :

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Exécution. Depuis la racine du projet :

Commande	Résultat
`.venv/bin/python section1_kmeans.py`	Exercices 1 à 3, figures `ex3_*.png`
`.venv/bin/python section1_sklearn.py`	Exercice 4
`.venv/bin/python section1_verifications.py`	Vérification de l'Exercice 1
`.venv/bin/python section2_hierarchique.py`	Section 2, figures `s2_*.png`
`.venv/bin/python section3_etude_de_cas.py`	Section 3, figures `s3_*.png`

Chaque script affiche les résultats cités dans ce compte rendu. Les tirages aléatoires de la Section 3 utilisent une graine fixe, donc les résultats et les figures sont les mêmes à chaque exécution.

1 Clustering k-means

1.1 Exercice 1 : implémentation de k-means

Notre implémentation est dans `section1_kmeans.py` et n'utilise que les modules `math` et `random` (Matplotlib sert seulement aux figures de l'Exercice 3). Un point est une liste de nombres (`[1]` en 1D, `[-2, 3]` en 2D) et tous les calculs se font coordonnée par coordonnée, donc le code marche quelle que soit la dimension.

```
kmeans(data, k, centroides_initiaux=None)
```

La fonction affiche les résultats intermédiaires et retourne la SSE. Elle procède ainsi :

1. **Initialisation** : si `centroides_initiaux` vaut `None`, on tire k points du jeu de données au hasard (`random.sample`).
2. **Affectation** : chaque point va au centroïde le plus proche (distance euclidienne). En cas d'égalité, il va au centroïde de plus petit indice.
3. **Mise à jour** : chaque centroïde devient la moyenne des points de son cluster. Un cluster vide garde son ancien centroïde.
4. **Arrêt** : on recommence les étapes 2 et 3 jusqu'à ce que l'affectation ne change plus.
5. **Erreur** : on retourne SSE $= \sum_{j=1}^{k} \sum_{x \in C_j} d(x, c_j)^2$.

Pour vérifier l'affichage, `section1_verifications.py` compare la sortie de notre fonction avec le texte de l'énoncé pour les deux exemples : elle est identique (SSE de 98.0 et de 2.5).

1.2 Exercice 2 : tests sur le jeu de données [[2], [4], [6], [12], [24], [30]]

(a) Centroïdes initiaux `[[2], [6]]`

```
Iteration 1
Centroid [2]  Points [[2], [4]]
Centroid [6]  Points [[6], [12], [24], [30]]
Iteration 2
Centroid [3.0]  Points [[2], [4], [6]]
Centroid [18.0]  Points [[12], [24], [30]]
Iteration 3
Centroid [4.0]  Points [[2], [4], [6], [12]]
Centroid [22.0]  Points [[24], [30]]
Iteration 4
Centroid [6.0]  Points [[2], [4], [6], [12]]
Centroid [27.0]  Points [[24], [30]]
Fin clustering, erreur =  74.0
```

(b) Centroïdes initiaux `[[12], [24]]`

```
Iteration 1
Centroid [12]  Points [[2], [4], [6], [12]]
Centroid [24]  Points [[24], [30]]
Iteration 2
Centroid [6.0]  Points [[2], [4], [6], [12]]
Centroid [27.0]  Points [[24], [30]]
Fin clustering, erreur =  74.0
```

Discussion. Les deux initialisations donnent le même clustering, $\{2, 4, 6, 12\}$ et $\{24, 30\}$, avec la même SSE (74.0) : aucune n'est meilleure que l'autre. Seul le nombre d'itérations change. Avec (b), les centroïdes 12 et 24 sont déjà de part et d'autre de la séparation, donc le bon clustering est trouvé dès la première itération. Avec (a), les deux centroïdes partent à gauche et le second doit se déplacer vers la droite (6, 18, 22 puis 27), d'où 4 itérations. Au passage, à l'itération 1 de (a), le point 4 est à égale distance de 2 et de 6 : il va dans le premier cluster d'après notre règle.

1.3 Exercice 3 : visualisations 2D

Nous avons ajouté trois paramètres optionnels :

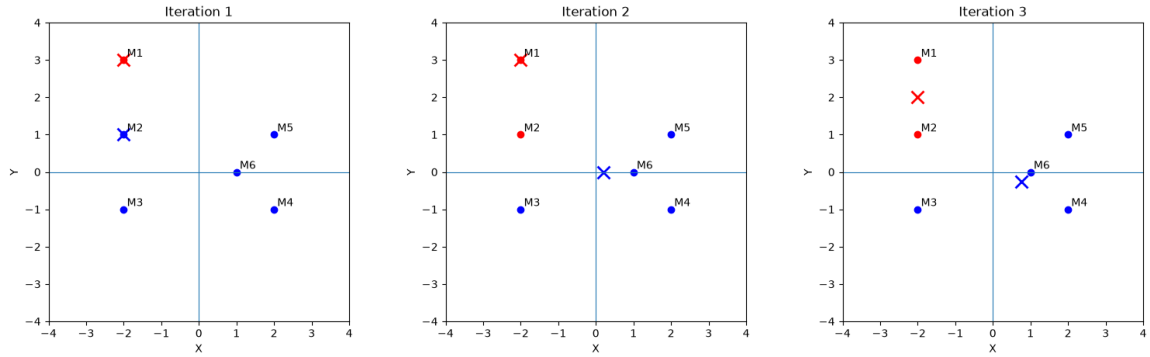
```
kmeans(data, k, centroides_initiaux=None, noms=None,
       prefixe_figures=None, retourner_clusters=False)
```

- `noms` : les noms des points, dans le même ordre que `data`, pour les annoter sur les figures ;
- `prefixe_figures` : si ce paramètre est donné, la figure de l'itération i est sauvegardée dans `<prefixe_figures>_iter<i>.png`. Nous sauvegardons les figures plutôt que de les afficher pour pouvoir les mettre dans le compte rendu ;
- `retourner_clusters` : si `True`, la fonction retourne (SSE, clusters finaux) au lieu de la SSE seule. Il sert à l'Exercice 4 et à l'étude de cas.

Avec les valeurs par défaut, la fonction se comporte comme à l'Exercice 1. Sur chaque figure, les points sont des • annotés et les centroïdes des ×, de la couleur de leur cluster (rouge pour le premier, bleu pour le second). Nous avons pris un cadre de -4 à 4 au lieu de -3 à 3 pour que M1 ne soit pas collé au bord.

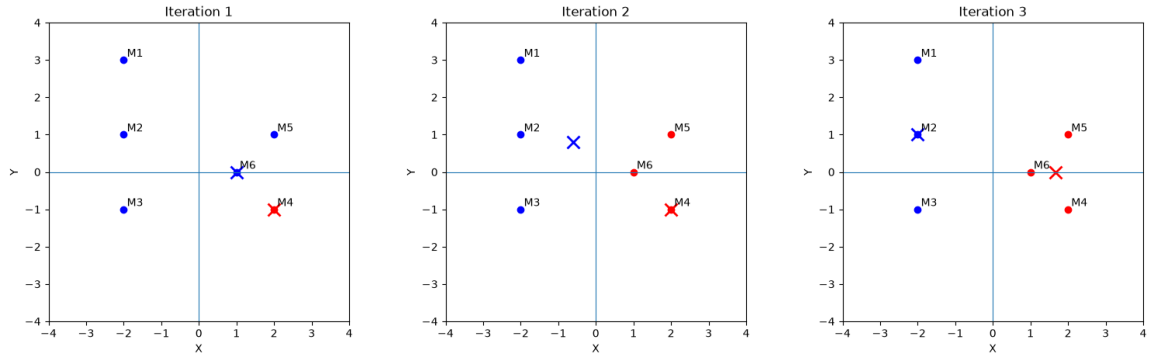
Les figures sont générées par `.venv/bin/python section1_kmeans.py`, avec ces deux appels (extrait simplifié) :

```
M = [[-2, 3], [-2, 1], [-2, -1], [2, -1], [2, 1], [1, 0]]
noms_M = ["M1", "M2", "M3", "M4", "M5", "M6"]
kmeans(M, 2, [M[0], M[1]], noms=noms_M,
       prefixe_figures="figures/ex3_init_M1_M2")   # M1 et M2
kmeans(M, 2, [M[3], M[5]], noms=noms_M,
       prefixe_figures="figures/ex3_init_M4_M6")   # M4 et M6
```



(a) Itération 1 (b) Itération 2 (c) Itération 3

FIGURE 1 – Centroïdes initiaux M1 et M2. SSE finale : 15.5.



(a) Itération 1 (b) Itération 2 (c) Itération 3

FIGURE 2 – Centroïdes initiaux M4 et M6. SSE finale : 10.67.

Nous obtenons les mêmes séquences que dans l'énoncé.

1.4 Exercice 4 : comparaison avec SciKit-Learn

`section1_sklearn.py` lance d'abord le code de l'énoncé, qui affiche bien `[0 1 2 2 2]` et `[0 0 1 1 2]`.

Pourquoi ce sont les clusters de l'Exercice 1. `labels_` donne le numéro du cluster de chaque point de `data1`, dans l'ordre. Deux points sont dans le même cluster s'ils ont le même numéro. `[0 1 2 2 2]` correspond donc à {1}, {2}, {18, 20, 31}, le résultat obtenu avec `[[1], [2], [18]]`, et `[0 0 1 1 2]` à {1, 2}, {18, 20}, {31}, obtenu avec `[[18], [20], [31]]`. Les numéros eux-mêmes n'ont pas d'importance, seul compte quels points ont le même.

Exercices 2 et 3. Le script refait ensuite tous les clusterings avec `KMeans(n_clusters=k, n_init=1, init=array(...))` et les compare aux nôtres, sans tenir compte de la numérotation des clusters. Il compare aussi notre SSE à `inertia_` (avec `math.isclose`, à cause des arrondis).

Exercice	Centroïdes initiaux	SSE	Mêmes clusters	Même SSE
1	`[[1], [2], [18]]`	98.0	OUI	OUI
1	`[[18], [20], [31]]`	2.5	OUI	OUI
2 (a)	`[[2], [6]]`	74.0	OUI	OUI
2 (b)	`[[12], [24]]`	74.0	OUI	OUI
3	M1, M2	15.5	OUI	OUI
3	M4, M6	10.67	OUI	OUI

SciKit-Learn trouve les mêmes clusters que nous dans les six cas.

2 Clustering hiérarchique

Le code est dans `section2_hierarchique.py`. La fonction `dendrogramme` lance le clustering avec `linkage` (méthode `"single"` ou `"complete"`) puis dessine le dendrogramme avec `dendrogram`, deux fonctions de `scipy.cluster.hierarchy`. La fonction `figure_single_complete` met les deux dendrogrammes côte à côte et sauvegarde la figure. Le script affiche aussi les fusions successives.

2.1 Matrice de distance

La matrice est donnée à `linkage` au format compact 1D, c'est-à-dire les valeurs au-dessus de la diagonale lues ligne par ligne :

```
data = [0.1, 0.9, 0.35, 0.8, 0.3, 0.4, 0.5, 0.6, 0.7, 0.2]
```

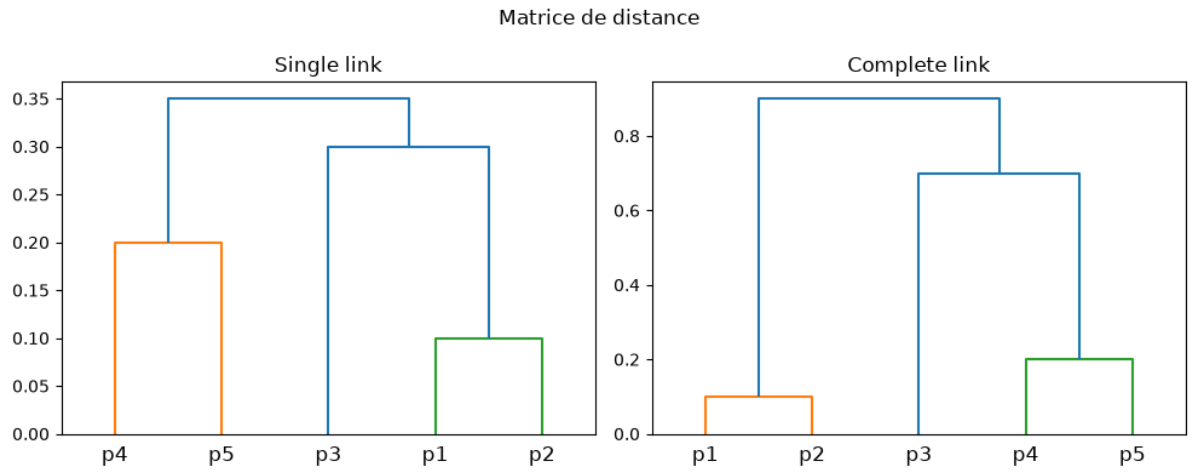


FIGURE 3 – Dendrogrammes obtenus avec la matrice de distance.

Nous obtenons les mêmes dendrogrammes que l'énoncé. Les deux premières fusions sont identiques dans les deux méthodes ($\{p_1, p_2\}$ à 0.1, puis $\{p_4, p_5\}$ à 0.2), car entre deux points seuls il n'y a qu'une distance possible. La différence vient de p_3 :

- en single link, la distance entre deux clusters est la plus petite distance entre leurs points. p_3 est à $\min(0.9, 0.3) = 0.3$ de $\{p_1, p_2\}$ et à $\min(0.6, 0.7) = 0.6$ de $\{p_4, p_5\}$, il rejoint donc $\{p_1, p_2\}$;
- en complete link, c'est la plus grande distance : $\max(0.9, 0.3) = 0.9$ contre $\max(0.6, 0.7) = 0.7$, donc p_3 rejoint $\{p_4, p_5\}$.

Pour la même raison, la dernière fusion se fait à 0.35 en single link ($d(p_1, p_4)$, la plus petite distance entre les deux groupes) et à 0.9 en complete link ($d(p_1, p_3)$, la plus grande).

2.2 Matrice de similarité

Il faut d'abord convertir la similarité en distance avec distance = 1−similarité. Nous calculons `1 - sim` sur la matrice 5×5, puis `squareform` (de `scipy.spatial.distance`) la met au format compact. Nous avons choisi `squareform` parce qu'elle vérifie en plus que la matrice est symétrique avec une diagonale nulle, ce qui détecte une erreur de recopie. On obtient :

```
[0.9, 0.59, 0.45, 0.65, 0.36, 0.53, 0.02, 0.56, 0.15, 0.24]
```

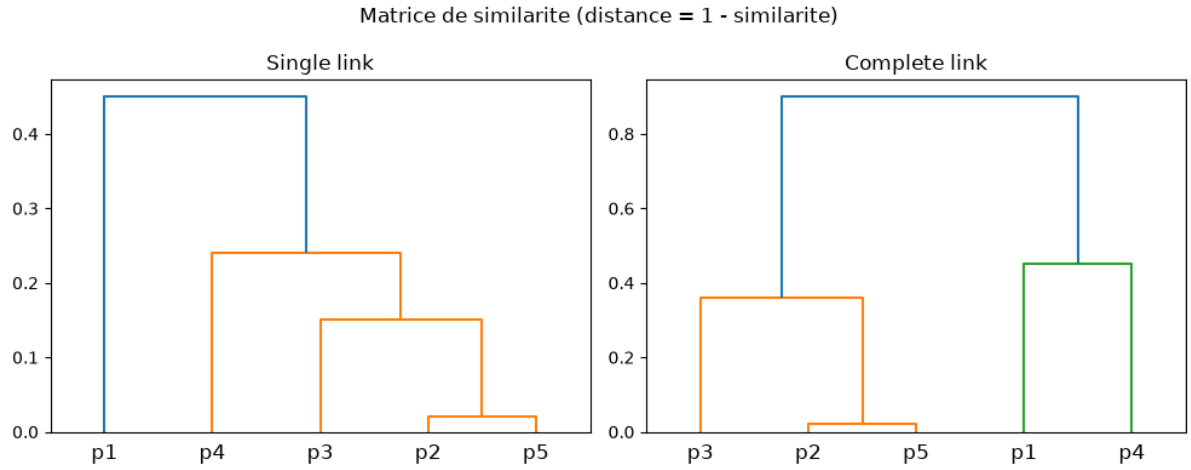


FIGURE 4 – Dendrogrammes obtenus avec la matrice de similarité convertie en distances.

Nous obtenons aussi les dendrogrammes de l'énoncé. En single link, p_2 et p_5 fusionnent d'abord (0.02), puis p_3 (0.15), p_4 (0.24) et enfin p_1 (0.45) rejoignent ce cluster un par un. C'est l'effet de chaîne du single link : il suffit d'être proche d'un seul point du cluster pour y entrer. Par exemple, p_4 entre grâce à $d(p_4, p_5) = 0.24$, alors qu'il est à 0.53 de p_2 et à 0.56 de p_3. En complete link, p_4 est à 0.56 de ce cluster (sa plus grande distance), donc il se regroupe plutôt avec p_1 (0.45). On obtient deux groupes plus compacts, $\{p_2, p_3, p_5\}$ et $\{p_1, p_4\}$, qui ne fusionnent qu'à 0.9.

3 Étude de cas

Le code est dans `section3_etude_de_cas.py`.

3.1 Exercice 1 : préparation des données

Lecture du fichier. La fonction `lire_donnees` retourne la liste `countries` des 167 noms de pays (pour étiqueter les figures) et la liste `data_country` de 167 listes de 9 nombres. Nous utilisons le module `csv` et non un simple `split(",")`, car quatre noms de pays contiennent une virgule et sont entre guillemets dans le fichier (par exemple `"Congo, Dem. Rep."`). Le script affiche le début de `data_country`, qui est bien celui de l'énoncé :

```
[[90.2, 10.0, 7.58, 44.9, 1610.0, 9.44, 56.2, 5.82, 553.0],
 [16.6, 28.0, 6.55, 48.6, 9930.0, 4.49, 76.3, 1.65, 4090.0], ...
```

ACP. La fonction `preparer_donnees` reprend le squelette de l'énoncé. Nous avons dû remplacer la ligne `eigenvectors = eig(R)[1]` par :

```
valeurs_propres, eigenvectors = eig(R)
# 1) R est symetrique reelle : ses valeurs propres sont reelles
assert np.allclose(valeurs_propres.imag, 0) and np.allclose(eigenvectors.imag, 0)
valeurs_propres, eigenvectors = valeurs_propres.real, eigenvectors.real
# 2) eig ne trie pas les valeurs propres : ordre decroissant
ordre = argsort(valeurs_propres)[::-1]
valeurs_propres, eigenvectors = valeurs_propres[ordre], eigenvectors[:, ordre]
# 3) le signe d'un vecteur propre est arbitraire : on prend celui de l'enonce
for c in range(2):
    if matmul(Z[0], eigenvectors[:, c]) * REFERENCE_ENONCE[c] < 0:
        eigenvectors[:, c] = -eigenvectors[:, c]
```

1. Chez nous, `eig` renvoie des nombres complexes, avec une partie imaginaire nulle. Nous gardons la partie réelle après avoir vérifié qu'elle est bien nulle.
2. `eig` ne range pas forcément les valeurs propres de la plus grande à la plus petite. Nous les trions pour que les colonnes 0 et 1 soient bien les deux composantes principales.
3. Si v est un vecteur propre, $-v$ aussi, et `eig` peut renvoyer l'un ou l'autre. Cela retourne la figure mais ne change pas les distances. Nous prenons le signe qui donne le même premier point que l'énoncé, pour avoir des figures orientées comme les siennes.

Le début de `data_reduced` est bien celui de l'énoncé : `[[2.91, -0.1], [-0.43, 0.59], ...]`. Les deux composantes expliquent 46.0 % et 17.2 % de la variance, soit 63.1 % : la projection en 2D perd donc une partie de l'information.

3.2 Exercice 2 : clustering et interprétation

Clustering k-means

Nous appliquons la fonction `kmeans` de la Section 1 à `data_reduced`, avec des centroïdes initiaux tirés au hasard. Avec 167 points, l'affichage des itérations est illisible, donc le script le masque et n'affiche qu'un résumé. Pour que le résultat soit reproductible, nous fixons la graine du tirage (`random.seed`). Nous avons choisi les graines 0 pour $k = 2$ et 13 pour $k = 3$ car elles redonnent les clusterings des Figures 1 et 2 de l'énoncé.

k	Graine	Itérations	SSE	Taille des clusters	Figure
2	0	14	503.12	89 (rouge), 78 (bleu)	5
3	13	8	373.47	41 (rouge), 48 (bleu), 78 (orange)	6

Aurait-on pu obtenir des clusterings très différents ? Pour le savoir, le script relance k-means avec les graines 0 à 199 et compte les clusterings obtenus :

k	Tirages	SSE	Clustering obtenu
2	107	503.12	celui de la Figure 5
2	85	503.02	le même, sauf Azerbaijan et Dominican Republic
2	8	762.70	Ireland, Luxembourg, Malta et Singapore seuls
3	56	373.47	celui de la Figure 6
3	144	354.52 à 415.12	29 autres clusterings

Pour $k = 2$, le résultat est presque toujours le même. Pour $k = 3$ en revanche, nous obtenons 30 clusterings différents et celui de l'énoncé ne sort que 56 fois sur 200. Certains sont proches (tailles 40, 47 et 80 par exemple), d'autres très différents (23, 70 et 74 pays, ou un cluster qui ne contient que Luxembourg, Malta et Singapore). Donc oui, surtout pour $k = 3$, nous aurions pu obtenir des clusterings assez différents de ceux de l'énoncé.

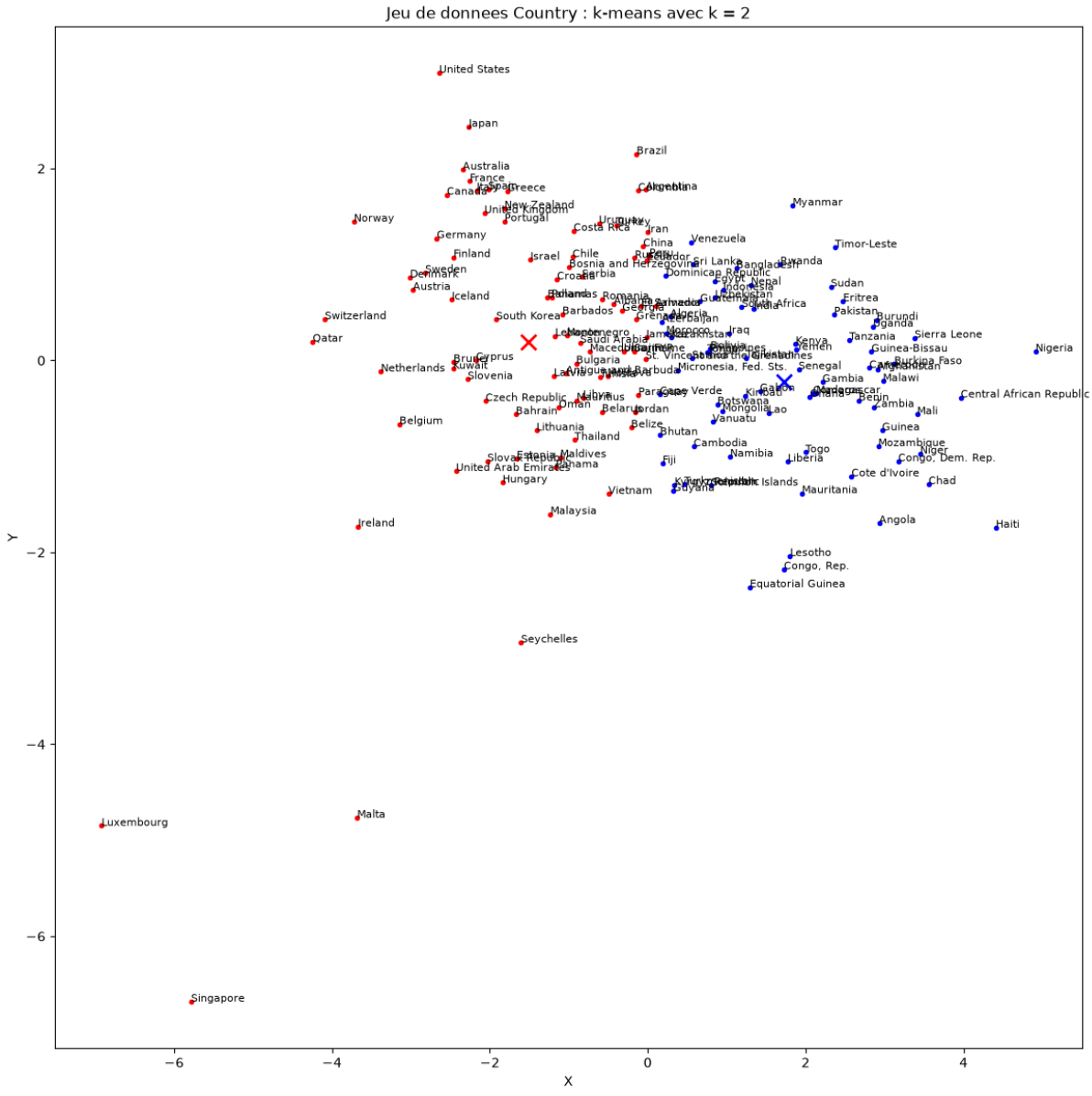


FIGURE 5 – k-means avec $k = 2$ sur `data_reduced`.

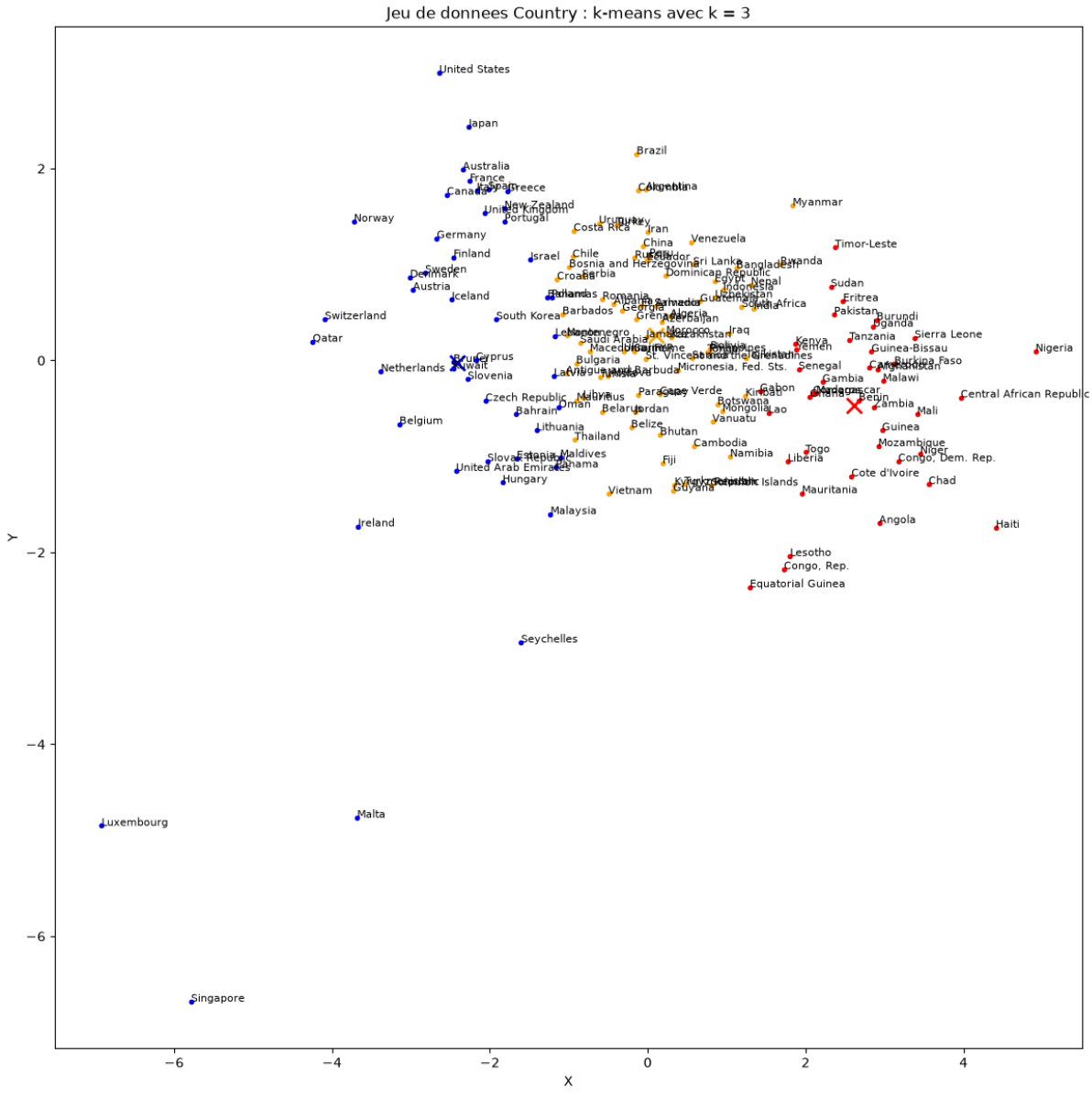


FIGURE 6 – k-means avec $k = 3$ sur `data_reduced`.

Clustering hiérarchique

Nous réutilisons la fonction `dendrogramme` de la Section 2 sans la modifier : avec une liste de points, `hierarchy.linkage` calcule lui-même les distances euclidiennes. Le dendrogramme est dessiné à l'horizontale pour que les noms des pays restent lisibles. Nous obtenons les mêmes dendrogrammes que les Figures 3 et 4 de l'énoncé (même ordre des pays).

Pour comparer avec k-means, le script coupe aussi chaque dendrogramme en 2 puis en 3 clusters (`hierarchy.fcluster`) :

Méthode	Clusters	Tailles	Petits clusters
Single link	2	2, 165	{Luxembourg, Singapore}
Single link	3	2, 164, 1	{Luxembourg, Singapore}, {Malta}
Complete link	2	3, 164	{Luxembourg, Malta, Singapore}
Complete link	3	3, 41, 123	{Luxembourg, Malta, Singapore}

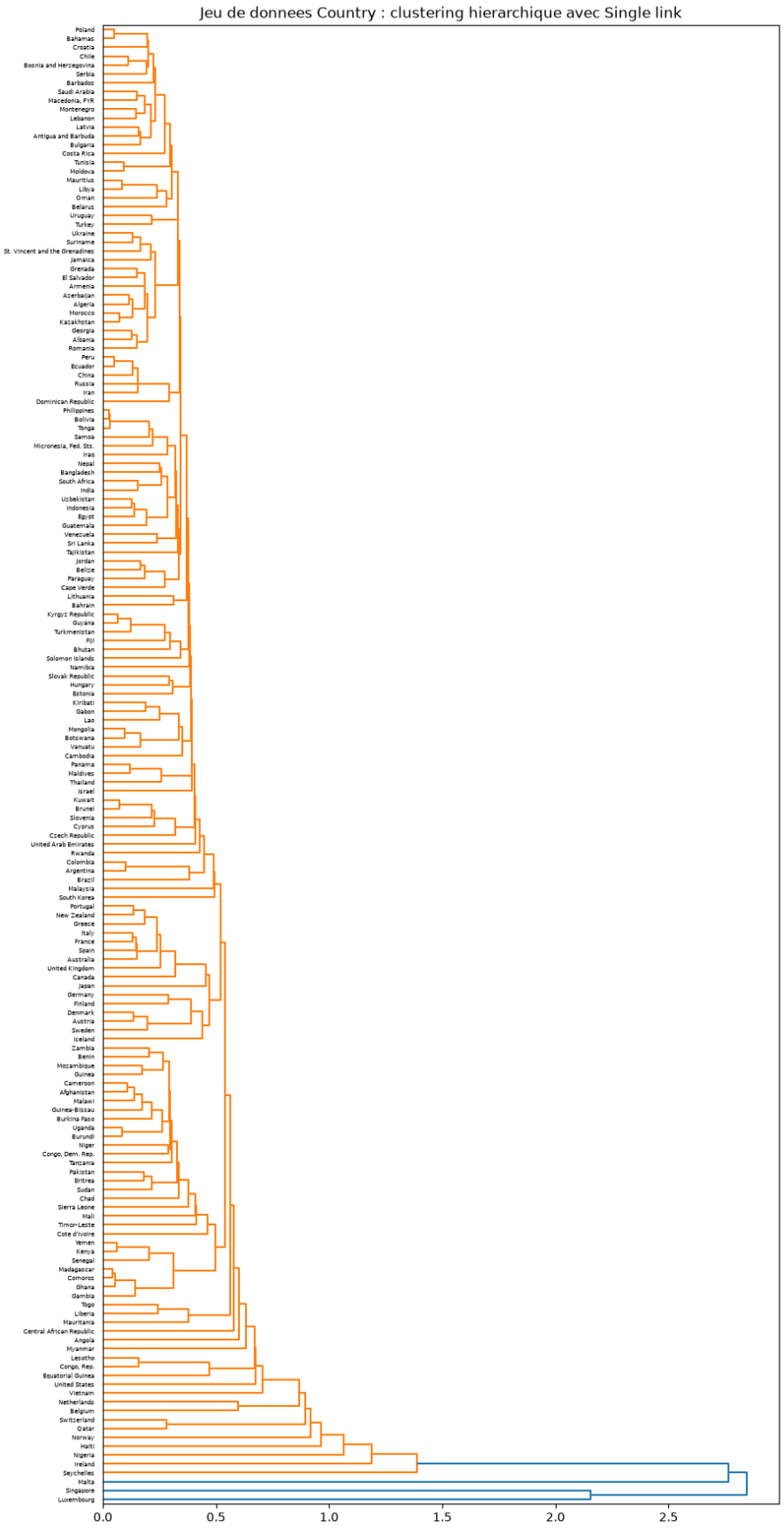


FIGURE 7 – Clustering hiérarchique single link sur `data_reduced`.

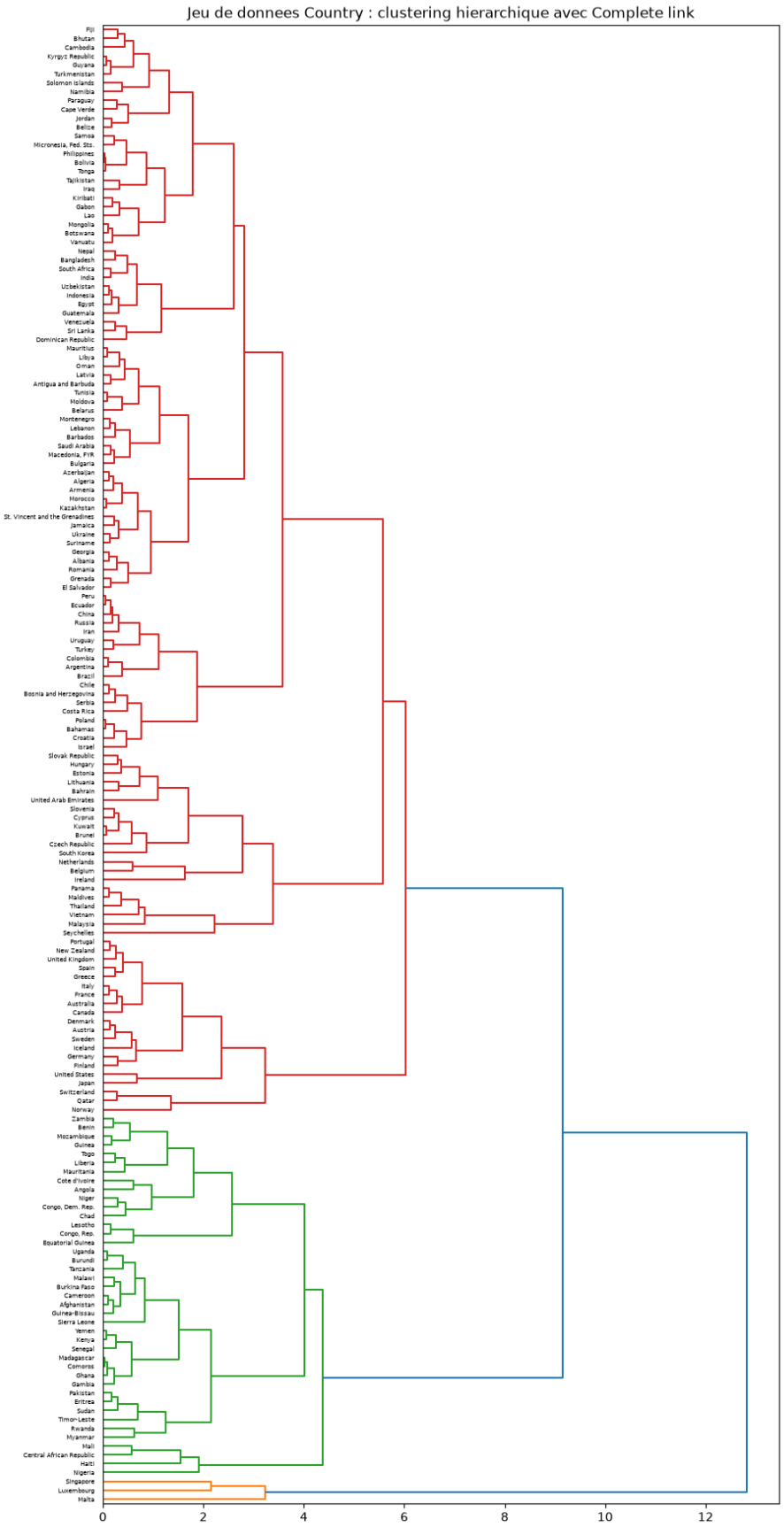


FIGURE 8 – Clustering hiérarchique complete link sur `data_reduced`.

Interprétation

Les axes. Le script affiche la corrélation de chaque variable avec les axes. L'axe X est surtout lié à la mortalité infantile (+0.85), à la fécondité (+0.82), à l'espérance de vie (−0.87), au revenu (−0.81) et au PIB par habitant (−0.80) : il mesure le niveau de développement, les pays pauvres étant à droite. L'axe Y est surtout lié aux importations (−0.84) et aux exportations (−0.76) en % du PIB : les pays qui commercent beaucoup avec l'étranger sont en bas.

Équilibre et sens des clusters k-means. Moyennes de quelques variables dans chaque cluster (affichées par le script) :

	Cluster	child_mort	income	gdpp	life_expec	total_fer	exports
$k = 2$	rouge (89)	10.4	27 691	22 216	77.0	1.9	49.9
	bleu (78)	70.1	5 111	2 408	63.2	4.2	31.1
$k = 3$	rouge (41)	98.6	3 311	1 585	59.0	5.2	28.9
	orange (78)	26.2	10 578	5 326	71.4	2.5	35.7
	bleu (48)	6.3	39 631	35 096	79.0	1.8	60.3

Avec $k = 2$, les clusters sont équilibrés : pays développés ou intermédiaires d'un côté, pays en développement de l'autre. Mais le cluster rouge mélange des pays très différents, comme Norway et Vietnam. Avec $k = 3$, les clusters restent assez équilibrés et correspondent à trois niveaux de développement : pauvre (rouge), intermédiaire (orange) et riche (bleu). C'est le clustering qui nous semble avoir le plus de sens.

Proximité sur la carte du monde. Pour $k = 3$, les clusters ressemblent à de grandes régions :

- rouge : 35 des 41 pays sont en Afrique, les autres sont Afghanistan, Haiti, Lao, Pakistan, Timor-Leste et Yemen ;
- bleu : Europe de l'Ouest et du Nord, une partie de l'Europe centrale, United States, Canada, Japan, South Korea, Australia, New Zealand et les pays du Golfe sauf Saudi Arabia ;
- orange : presque toute l'Amérique latine, la plupart des pays des Balkans et de l'ex-URSS, l'Afrique du Nord, l'Afrique australe et une grande partie de l'Asie (China, India, Indonesia…).

La géographie n'entre pas dans le clustering, mais des pays voisins ont souvent un niveau de développement proche. Comme on pouvait s'y attendre, les pays nordiques sont côte à côte sur la Figure 6, tout comme France, Italy et Spain, ou les pays du Sahel (Mali, Niger, Chad). Certaines proximités nous ont surpris :

- Haiti est dans le cluster rouge, à côté de Chad et Niger, alors que Dominican Republic, sur la même île, est dans le cluster orange. Dans les données, Haiti a la mortalité infantile la plus élevée (208) et l'espérance de vie la plus faible (32.1 ans) des 167 pays.
- Equatorial Guinea et Gabon sont dans le cluster des pays pauvres malgré un revenu élevé (33 700 et 15 400), car leur mortalité infantile et leur espérance de vie sont celles des pays pauvres.
- Malaysia, Maldives, Panama et Seychelles sont dans le cluster riche avec un PIB par habitant de 7 100 à 10 800, alors que Saudi Arabia (19 300) est dans le cluster orange. Ces quatre pays exportent beaucoup (70 à 94 % du PIB), ce qui les place en bas de la figure, plus près du centroïde bleu.
- Singapore est très loin de Luxembourg et de Malta sur la carte, mais les trois pays sont proches sur la figure, isolés en bas à gauche. Avec Ireland, ce sont les pays qui exportent le plus (103 à 200 % du PIB) : leur proximité vient de leur économie, pas de la géographie.

Clusterings hiérarchiques. En single link, les clusters sont très déséquilibrés (2 et 165 pays) et n'ont pas de sens pour regrouper les pays : à cause de l'effet de chaîne, presque tous les pays sont reliés de proche en proche, et seuls les pays isolés se séparent. Le complete link donne un résultat plus intéressant : en 3 clusters, le cluster de 41 pays a 39 pays en commun avec le cluster rouge de k-means (seuls Myanmar, Rwanda, Gabon et Lao changent). Deux méthodes différentes trouvent donc presque le même groupe de pays pauvres. En revanche, il met ensemble les pays riches et intermédiaires (123 pays) et laisse Luxembourg, Malta et Singapore dans un cluster à part.

Conclusion. Sur ce jeu de données, k-means avec $k = 3$ donne le clustering le plus utile. Il faut quand même garder en tête que la projection ne garde que 63.1 % de la variance, et que l'axe Y rapproche les pays qui commercent beaucoup, quel que soit leur niveau de développement.